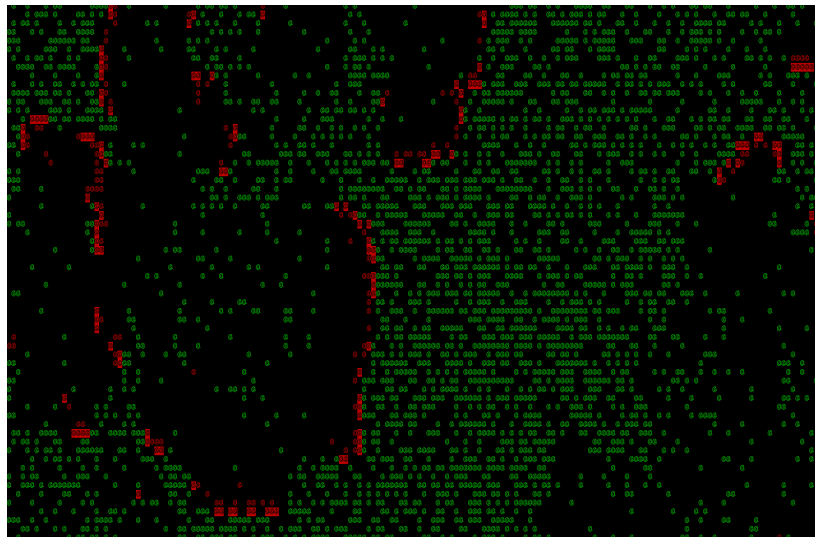


Programmation en C : Simulation de feu de forêt

INSA DE STRASBOURG
MÉCATRONIQUE 3 - FIP-MIK4

Emmanuel Roussel
emmanuel.roussel@isl.eu

2020-2021



Le programme `burning_cells.c` permet de simuler un feu de forêt en utilisant un certain nombre de règles simples.

Il s'agit d'un automate cellulaire. Le monde est représenté comme une grille de dimension finie, pouvant évoluer au cours du temps. Chaque cellule de cette grille peut prendre l'un des trois états suivants :

- vide (empty)
- arbre (tree)
- en feu (burning)

Le monde évolue de manière discrète en suivant les quatre règles suivantes, appliquées simultanément à toutes les cellules à chaque itération. Ces règles permettent de calculer l'état de chaque cellule au temps $t + 1$:

1. une cellule "en feu" au temps t devient "vide" au temps $t + 1$,
2. un arbre brûle si au moins une de ses huit cellules voisines est "en feu",
3. un arbre prend feu spontanément avec une probabilité p_{burn} (même si ses voisins ne sont pas en feu),
4. un arbre apparait dans une cellule vide avec une probabilité p_{tree} .

Le programme fourni est fonctionnel mais n'est pas complet : la règle n°2 n'est pas implémentée et le feu ne se propage donc pas. Vous le complèterez au fil des questions.

1 Tâches à réaliser

1. Installez les outils de compilation¹ :
 - Sous Linux et Mac, le compilateur GCC est installé par défaut.
 - Sous Windows, installez le logiciel MSYS2 qui permet de disposer d'une console et d'outils de développement UNIX sous Windows. Suivez pour cela les instructions sur le site [msys2.org](https://www.msys2.org), puis installez le compilateur GCC et l'outil make à l'aide de la commande :
`pacman -S gcc make.`
2. Compilez le programme avec GCC et assurez vous de son fonctionnement. Changez la probabilité p_{tree} d'apparition d'un arbre et la probabilité p_{burn} d'un feu, et vérifiez le comportement du programme.
3. Quelle variable est utilisée pour stocker l'état des cellules au temps t ? Et au temps $t + 1$?
4. Assurez vous de bien comprendre le fonctionnement du programme, puis complétez la fonction `check_neighbors` pour permettre la propagation du feu².
5. Modifiez le programme pour ajouter un nouvel état "cendre" en remplaçant la règle 1 par les deux nouvelles règles suivantes :
 - (a) une cellule "en feu" au temps t devient "cendre" au temps $t + 1$
 - (b) une cellule "cendre" au temps t devient "vide" au temps $t + 1$

Les autres règles (2 à 4) sont inchangées.

On pourra utiliser le code d'échappement `\033[00;31m@\033[m` pour l'affichage d'une cellule à l'état "cendre".

2 Annexes

Les "codes d'échappement" permettent d'envoyer des commandes particulières à la console. Ils permettent par exemple de contrôler la position du curseur, la couleur, etc.

En voici quelques exemples :

Code	Effet
<code>\033[H</code>	Place le curseur en haut à gauche de l'écran
<code>\033[2J</code>	Efface l'écran
<code>\033[32mhello\033[m</code>	Affiche le texte <code>hello</code> en vert (32) puis remet l'affichage par défaut (code <code>\033[m</code>)
<code>\033[31mhello\033[m</code>	Affiche le texte <code>hello</code> en rouge (31)
<code>\033[07;31mhello\033[m</code>	Affiche le texte <code>hello</code> en noir sur fond rouge (inversion grace à 07)

Les codes sont envoyés à la console à l'aide de la fonction `printf`. Voici un exemple de programme :

```
1  #include <stdio.h>
2
3  void main() {
4      printf("\033[31mhello\033[m");
5  }
```

1. En solution de secours, vous pourrez utiliser le compilateur en ligne OnlineGDB : https://www.onlinegdb.com/online_c_compiler

2. Attention à ne pas accéder à un élément hors des limites du tableau!